KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

TravelEase

A Smart Travel Planning & Tour Booking Platform (MERN Stack)

Project Documentation

Detail	Information
Project Link	https://mern-project-fgjm.vercel.app/
Team Number	1
Team GitHub Repository	https://github.com/Michel-28/MERN-PROJECT.git

Team Members

Student Name	Registration Number	Individual GitHub Link
BALAGANI VENKATA KARTHEEK	99230040484	https://github.com/Michel-28/MERN-PROJECT.git
AVULA RAMESWARA REDDY	99230040481	https://github.com/avularameswarareddy143-arch/MERN-PROJECT-2.git
BANKA SATHISH KUMAR REDDY	99230040487	https://github.com/satishreddy75997-maker/MERN-PROJECT.git
CHADUVULA LOKESWARA REDDY	99230040505	https://github.com/chaduvulalokeswarareddy2005-max/MERN
YEJARLA NANDA KUMAR	99230040461	https://github.com/Nandakumar-9/MERN-PROJECT
NALLANUKALA TRINATH KUMAR	99230040688	https://github.com/Trinath5094/MERN-PROJECT

PROJECT TRAINER / GUIDE

Ajusha Lalasan

Table of Contents

Section	Reference
1. Introduction	Section 1
2. Problem Statement & Objectives	Section 2
3. System Architecture (Booking & Payment Workflow)	Section 3
4. Implementation Details & Tech Stack	Section 4
5. Results and Testing	Section 5
6. Conclusion & Future Scope	Section 6

1. Introduction

Planning a trip usually involves several disconnected steps: finding a destination, comparing tour packages, working out a budget, booking, paying and keeping track of the itinerary. Travellers often handle these on separate websites, spreadsheets and messaging apps, which makes it hard to keep costs, dates and bookings consistent.

TravelEase is a full-stack web application built on the MERN stack (MongoDB, Express.js, React and Node.js) that brings these steps into one platform. Users can explore destinations, view tour packages with day-wise itineraries, register with email OTP verification, book a package for a chosen departure date, pay through a Razorpay-based payment flow, and download a booking receipt. A built-in trip planner lets users create their own trips with a day-wise itinerary, a category-wise budget and a packing checklist.

The platform also provides a separate administrator console, protected by role-based access control, where administrators manage destinations and packages, monitor bookings and verified revenue, and view registered users. Authentication uses JSON Web Tokens (JWT), passwords are hashed with bcrypt, and booking amounts are calculated on the server so that the client cannot alter prices.

2. Problem Statement & Objectives

> **Problem:** Travellers often have to use different tools for discovering destinations, booking tours, making online payments and planning budgets. Bookings made without proper payment verification, clear cancellation terms or availability checks can lead to double bookings, disputes over refunds and unclear costs. Tour operators, in turn, need a reliable way to track bookings, confirmed payments and customers in one place.

Objectives

- To develop a responsive full-stack web application for discovering destinations and tour packages, with search, filters and sorting.
- To implement secure user registration and login using email OTP verification, JWT-based sessions, hashed passwords and optional Google sign-in.

- To provide a complete booking and payment workflow with departure-date availability checks, server-side price calculation and Razorpay payment signature verification.
- To handle booking cancellation with a refund policy based on the number of days remaining before departure, and to generate a printable booking receipt.
- To give users a trip planner with a day-wise itinerary, category-wise budget tracking and a travel checklist.
- To provide an administrator console, protected by role-based access control, for managing destinations, packages, bookings and users.

3. System Architecture (Booking & Payment Workflow)

TravelEase follows a three-tier client–server architecture. A React single-page application communicates with an Express.js REST API (served under /api/v1), which reads and writes data in MongoDB through Mongoose. The API connects to external services for payments (Razorpay), email delivery (Nodemailer with Gmail) and Google sign-in. The main functionality can be understood through four connected workflows: authentication, booking and payment, cancellation and refund, and trip planning.

3.1 Overall Architecture

Layer	Description
Presentation Layer	React 19 single-page application built with Vite and styled with Tailwind CSS. React Router handles public, user-protected and admin-protected routes. Axios sends API requests and attaches the JWT as a Bearer token.
Application Layer	Node.js and Express.js REST API. Routes are grouped by module (auth, destinations, packages, bookings, payments, trips, favorites, admin). Middleware performs JWT verification (protect), role checks (authorize) and central error handling.
Data Layer	MongoDB accessed through Mongoose. Seven collections: User, Destination, TourPackage, Booking, Payment, Trip and Favorite. The database can run locally or on MongoDB Atlas.
External Services	Razorpay (payment orders and signature verification), Google OAuth (sign-in) and Gmail through Nodemailer (OTP emails).

3.2 User Registration & Authentication

1. The user submits full name, email, phone number, password and confirmation password on the registration page. The server validates the email format, the minimum password length (6 characters) and that both passwords match.
2. The server hashes the password with bcryptjs and creates an unverified account. The role is always set to “user”, so a public request cannot create an administrator.
3. A 6-digit OTP is generated using Node’s crypto module, stored in hashed form with a 5-minute expiry, and emailed to the user through Nodemailer.
4. The user enters the OTP on the verification page. The server allows a maximum of 5 attempts per OTP, checks expiry, and on success marks the email as verified and issues a JWT. A new OTP can be requested after a 60-second cooldown.

5. On login, the server verifies the credentials and returns a JWT (valid for 7 days by default). The client stores the token and sends it with each API request. Users can also sign in with Google, and administrators use a separate admin login that checks the “admin” role.
6. Protected API routes run the protect middleware, which verifies the token and the account status. Administrative routes additionally run authorize(“admin”).

3.3 Tour Booking & Payment Workflow

1. The user browses destinations and packages, opens a package and proceeds to the checkout page, where a departure date and the number of travellers are selected.
2. The server checks availability for that departure date (package capacity, default 20, minus travellers already booked or pending), calculates the total as price per traveller × number of travellers, and creates a booking with the status “Pending Payment”. A reference code in the form TE-XXXXXX is generated.
3. The client requests a payment order. The server creates a Razorpay order for the verified amount. If valid Razorpay keys are not configured, the server creates a test-gateway order so the flow can be demonstrated offline.
4. The user pays through the gateway window (UPI, NetBanking or Card). In test mode, the user can simulate a successful or a failed payment.
5. The server verifies the payment. In Razorpay mode it recomputes the HMAC SHA-256 signature of “order_id|payment_id” using the key secret and compares it with the signature received. In test mode it validates the test payment ID format.
6. On success, the payment status becomes “paid” and the booking becomes “Confirmed”. The user is taken to the confirmation page, which shows the reference code, transaction ID, payment method and an itemised receipt that can be printed or saved as PDF. A repeated verification request returns the existing confirmation instead of creating a duplicate. On failure or cancellation, the booking is marked “Payment Failed”.

3.4 Cancellation & Refund Policy

A user can cancel a booking from the My Bookings page. If the booking was paid, the refund percentage is decided by the time remaining before departure. The cancellation reason, time, refund amount and a test refund transaction ID are stored in both the Booking and Payment records.

Time Before Departure	Refund	Result
More than 7 days	100%	Full refund of the amount paid
2 to 7 days	70%	Partial refund
Less than 2 days (48 hours)	0%	Non-refundable

3.5 Trip Planner & Budget Management

1. The user creates a trip by entering a trip name, destinations, start and end dates, number of travellers and a target budget.
2. In the itinerary tab, activities are added day-wise with time, location, notes and estimated cost. The itinerary can be viewed as a day-wise list or as a timeline, and activities can be reordered.

3. In the budget tab, expenses are entered under six categories: transportation, accommodation, food, activities, shopping and other. The server calculates the total planned expense before saving; the interface shows the remaining budget and a category breakdown chart.
4. The checklist tab provides a default travel checklist (documents, packing, transport, accommodation, electronics and planning) that the user can tick off or extend.

3.6 Administrator Management

1. The administrator signs in through the dedicated admin portal; the server confirms the "admin" role stored in the database.
2. The dashboard shows totals for destinations, packages, users and bookings, verified paid revenue, refunded value and the booking status distribution (Confirmed, Pending Payment, Payment Failed, Cancelled).
3. The bookings page supports search by reference, customer, package or destination, and filters by booking and payment status. A detail view shows payment and cancellation information.
4. Administrators can also view the registered users directory and create, edit or delete destinations and tour packages.

3.7 Platform Workflow Overview

Stage	Description
User Input	Registration details, OTP, package, departure date, traveller count, payment action, trip and budget details
React Client	Validates input, calls the REST API through Axios and attaches the JWT
Express Middleware	Verifies the JWT (protect), checks the role (authorize) and handles errors centrally
Controllers	Apply business rules: availability check, price calculation, signature verification, refund calculation
MongoDB (Mongoose)	Stores users, destinations, packages, bookings, payments, trips and favourites
External Services	Razorpay for payment orders, Gmail (Nodemailer) for OTP emails, Google for sign-in
Application Output	Booking confirmation, receipt, booking history, trip plans and the administrator dashboard

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend	React 19, Vite, Tailwind CSS, React Router DOM 7	Building the responsive user interface and client-side routing
UI Support	Axios, Lucide React, React Hot Toast	API communication, icons and notifications
Backend	Node.js, Express.js 4	REST API, middleware and business logic
Database	MongoDB, Mongoose 8 (local or Atlas)	Storing users, packages, bookings, payments and trips

Component	Technology Used	Purpose
Authentication	JWT (jsonwebtoken), bcryptjs, Google OAuth (@react-oauth/google, google-auth-library)	Secure sign-in, password hashing and role-based access
Email	Nodemailer (Gmail)	Sending OTP verification emails
Payments	Razorpay SDK (test mode) with test-gateway fallback	Creating payment orders and verifying signatures
Testing	Node.js test scripts (test_e2e.js and others)	Automated API tests against the running application
Deployment	Vercel configuration for the client (vercel.json), MongoDB Atlas for the database	Hosting the frontend and cloud database

4.1 Main Source Files

Source File	Responsibility
server/src/server.js, app.js	Starts the server, connects to the database and configures CORS, JSON parsing and routes.
server/src/middleware/auth.js	Contains the protect, authorize and optionalProtect middleware.
server/src/controllers/authController.js	Registration, OTP verification and resend, login, admin login, Google sign-in and profile update.
server/src/controllers/bookingController.js	Booking creation with availability check and server-side pricing, and cancellation with refund calculation.
server/src/controllers/paymentController.js	Payment order creation, signature verification, failure handling and receipt generation.
server/src/controllers/adminController.js	Dashboard statistics, booking and user listings for administrators.
server/src/controllers/tripController.js	Trip creation, update, deletion and budget calculation.
server/src/utils/sendEmail.js	Sends OTP emails through Nodemailer.
server/src/seed/seed.js	Populates destinations, packages, sample data and the administrator and demo accounts.
client/src/App.jsx	Defines all routes with ProtectedRoute and AdminRoute guards.
client/src/context/AuthContext.jsx	Manages login state, token storage and logout.
client/src/services/api.js	Axios instance and all API helper functions.
client/src/pages/BookingFlow.jsx	Checkout page and payment gateway interaction.
client/src/pages/TripDetail.jsx	Itinerary, budget and checklist tabs of the trip planner.
client/src/pages/admin/	Administrator dashboard, bookings, users, destinations and packages pages.

4.2 Database Design

Collection	Key Fields
User	name, email (unique), hashed password, phone, role (user / admin), accountStatus, authProvider (local / google), isEmailVerified, OTP details (hashed code, expiry, attempts)
Destination	name, slug, country, region, category, description, images, attractions, bestTimeToVisit, suggestedDuration, estimatedDailyBudget, travelTips
TourPackage	title, slug, destination (reference), durationDays / durationNights, pricePerTraveler, inclusions, exclusions, day-wise itinerary, departureDates, capacity
Booking	bookingReference, user, package and destination snapshots, departureDate, travelerCount, totalAmount, status, paymentStatus, cancellation details (reason, refund amount, refund transaction ID)
Payment	booking, user, provider (razorpay / test_gateway), gatewayOrderId, gatewayPaymentId, gatewaySignature, amount, status, verifiedAt, refund details
Trip	email, name, destinations, startDate, endDate, travelers, targetBudget, activities, budgetExpenses (six categories), totalPlannedExpenses, checklist, linkedBooking
Favorite	email, destination (unique combination of both)

4.3 REST API Summary

All endpoints are served under the base path /api/v1.

Module	Endpoints	Access
Auth	POST /auth/register, /auth/verify-otp, /auth/resend-otp, /auth/login, /auth/admin-login, /auth/google; GET /auth/me; PUT /auth/profile	Public (me and profile need login)
Destinations	GET /destinations, /destinations/featured, /destinations/:idOrSlug; POST, PATCH, DELETE for management	Read: public; changes: admin
Packages	GET /packages, /packages/:idOrSlug, /packages/:idOrSlug/availability; POST, PATCH, DELETE for management	Read: public; changes: admin
Bookings	GET /bookings, GET /bookings/:id, POST /bookings, PATCH /bookings/:id/cancel	Logged-in user
Payments	POST /payments/create-order, /payments/verify, /payments/failure; GET /payments/receipt/:bookingId	Logged-in user
Trips	GET, POST /trips; GET, PATCH, DELETE /trips/:id	Trip planner users
Favorites	GET, POST, DELETE /favorites	Trip planner users
Admin	GET /admin/dashboard/stats, /admin/bookings, /admin/bookings/:id, /admin/users, /admin/destinations, /admin/packages	Admin only

4.4 Security Features

- Passwords are hashed with bcryptjs and are excluded from query results by default; OTP codes are also stored in hashed form.

- JWT-based sessions with server-side verification and an account-status check on every protected request.
- Role-based access control: administrative routes require the “admin” role, and public registration always creates a “user”.
- Email OTP verification with a 5-minute expiry, a maximum of 5 attempts and a 60-second resend cooldown.
- Server-side price calculation and capacity checks, so totals sent from the browser are never trusted.
- HMAC SHA-256 payment signature verification and idempotent payment confirmation to avoid duplicate bookings.
- Users can view only their own bookings; cross-user access returns 403 Forbidden.
- Secrets (database URL, JWT secret, payment and email credentials) are kept in environment variables and excluded from version control through .gitignore.

4.5 Setup & Running the Project

1. Install Node.js (v18 or higher) and MongoDB (local) or create a MongoDB Atlas cluster.
2. Create server/.env and client/.env from the provided .env.example files (database URL, JWT secret, admin credentials, Razorpay, Google and email settings).
3. Run “npm run install:all” in the project root to install dependencies for the root, client and server.
4. Run “npm run seed” to load sample destinations, packages and the administrator and demo accounts.
5. Run “npm run dev” to start the API (port 5000) and the React client (port 5173) together.

5. Results and Testing

The platform provides a complete end-to-end travel booking experience. A user can register and verify their email with an OTP, browse and filter destinations and packages, select a departure date, pay through the payment gateway, receive a confirmation with a printable receipt, and later cancel with an automatic refund calculation. Users can also plan their own trips with an itinerary, budget and checklist, while administrators monitor bookings, verified revenue and users from a separate console.

5.1 Functional Results

Module	Outcome
Authentication	Registration with email OTP, login, Google sign-in and a separate admin login work with JWT-based sessions.
Destinations & Packages	Public pages list destinations and tour packages with search, filters and sorting, and show full details and day-wise itineraries.
Booking & Payment	Bookings move from “Pending Payment” to “Confirmed” only after payment verification; failed payments are recorded.
Receipt & Cancellation	A confirmed booking has a printable receipt; cancellation applies the refund policy automatically.
Trip Planner	Users can create trips with a day-wise itinerary, a six-category budget with remaining-amount tracking and a checklist.

Module	Outcome
Admin Console	Administrators view statistics, manage bookings, users, destinations and packages.

5.2 Application Screenshots

The following figures show the main pages of the application. Each box below is a placeholder; replace it with the corresponding screenshot (Insert > Pictures) and resize it to the page width.

5.2.1 Public Pages

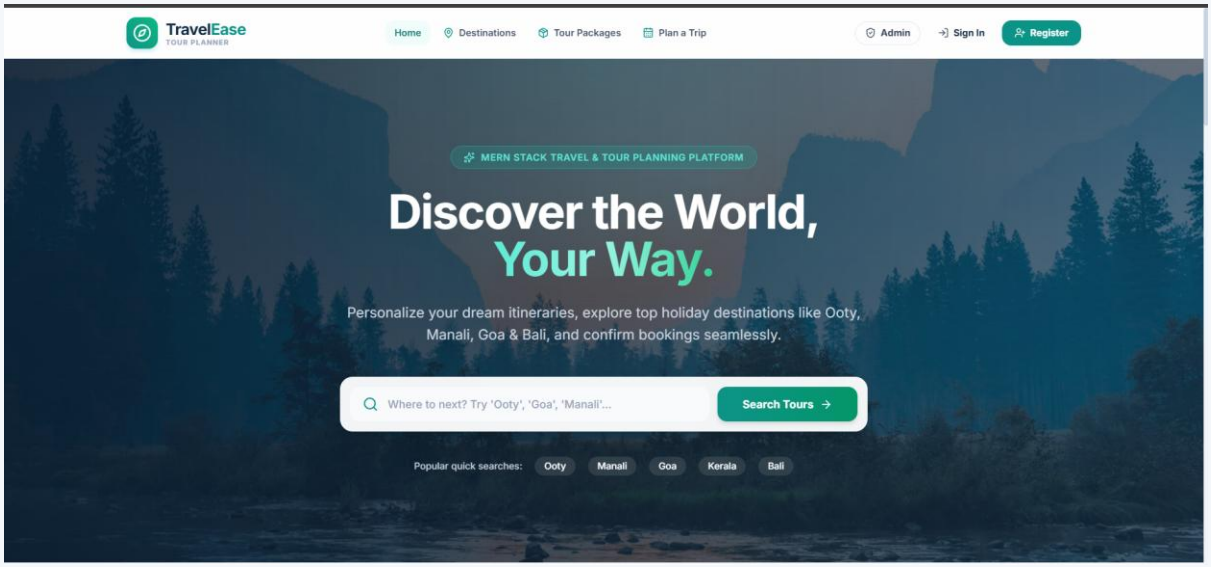


Figure 1: Home Page

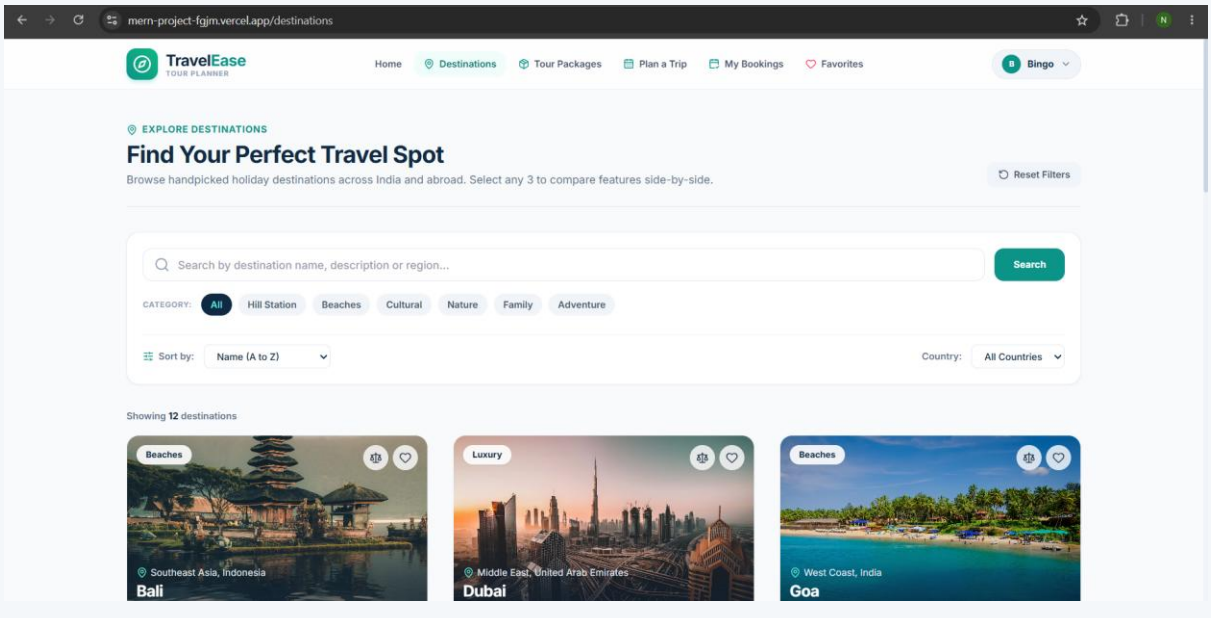


Figure 2: Destinations Listing

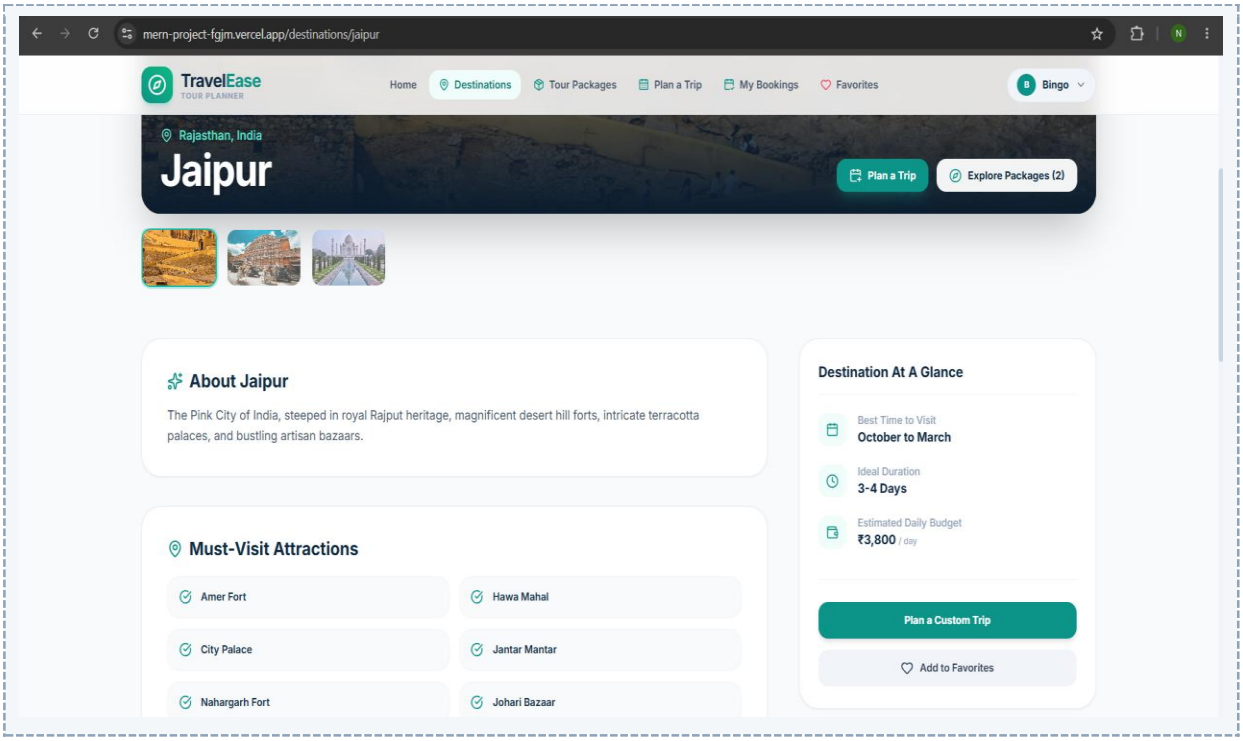


Figure 3: Destination Details

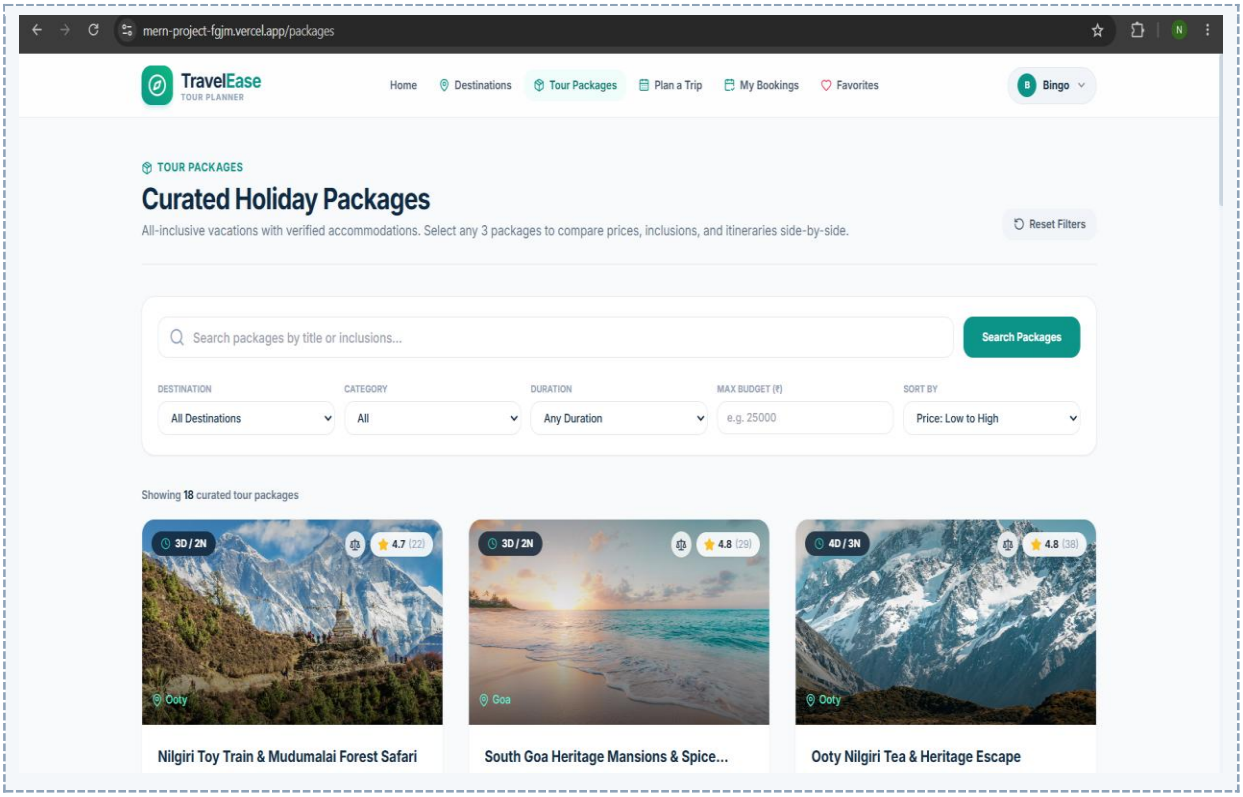


Figure 4: Tour Packages Listing

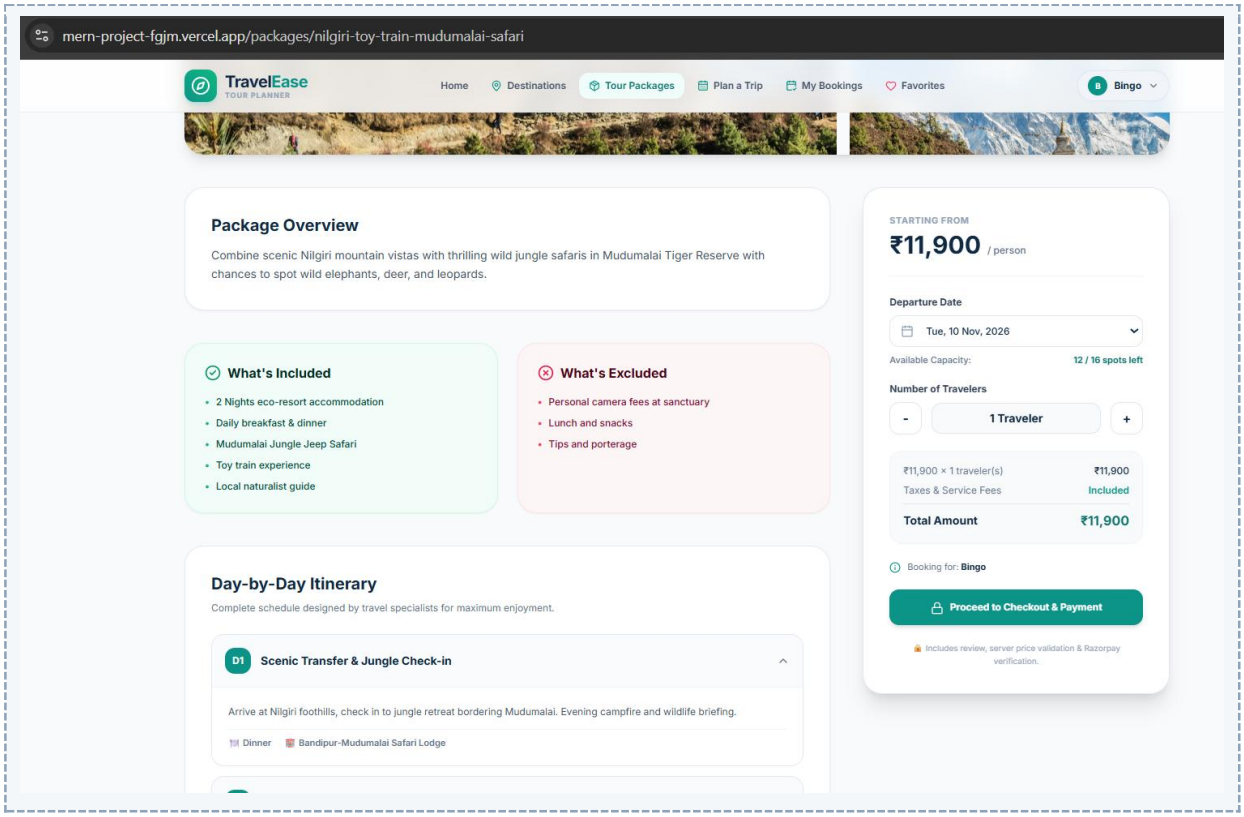


Figure 5: Package Details

5.2.2 Authentication Pages

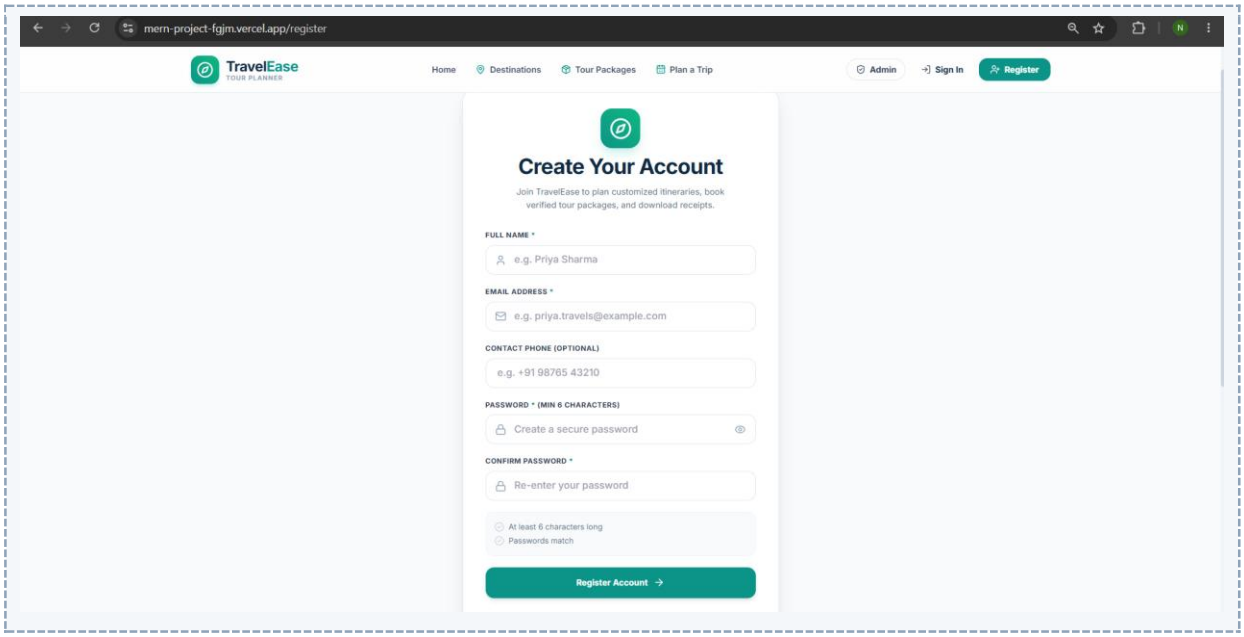


Figure 6: User Registration

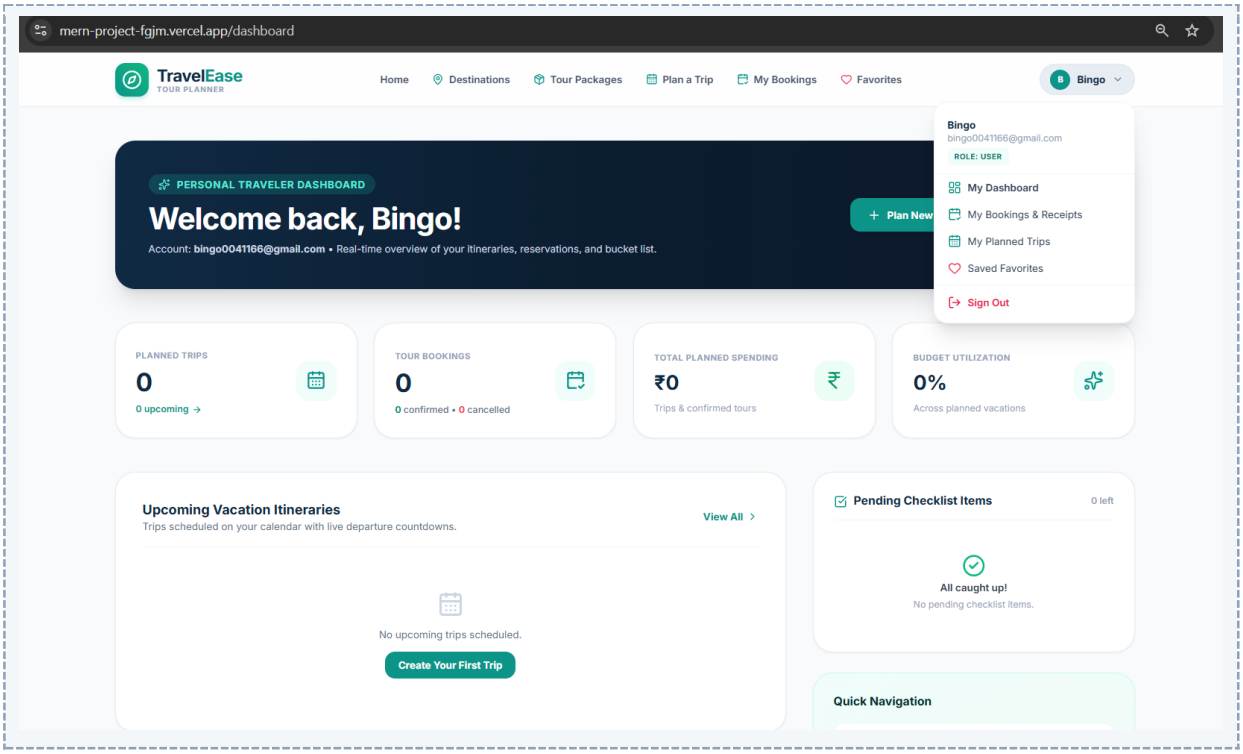


Figure 7: User Login

5.2.3 User Pages

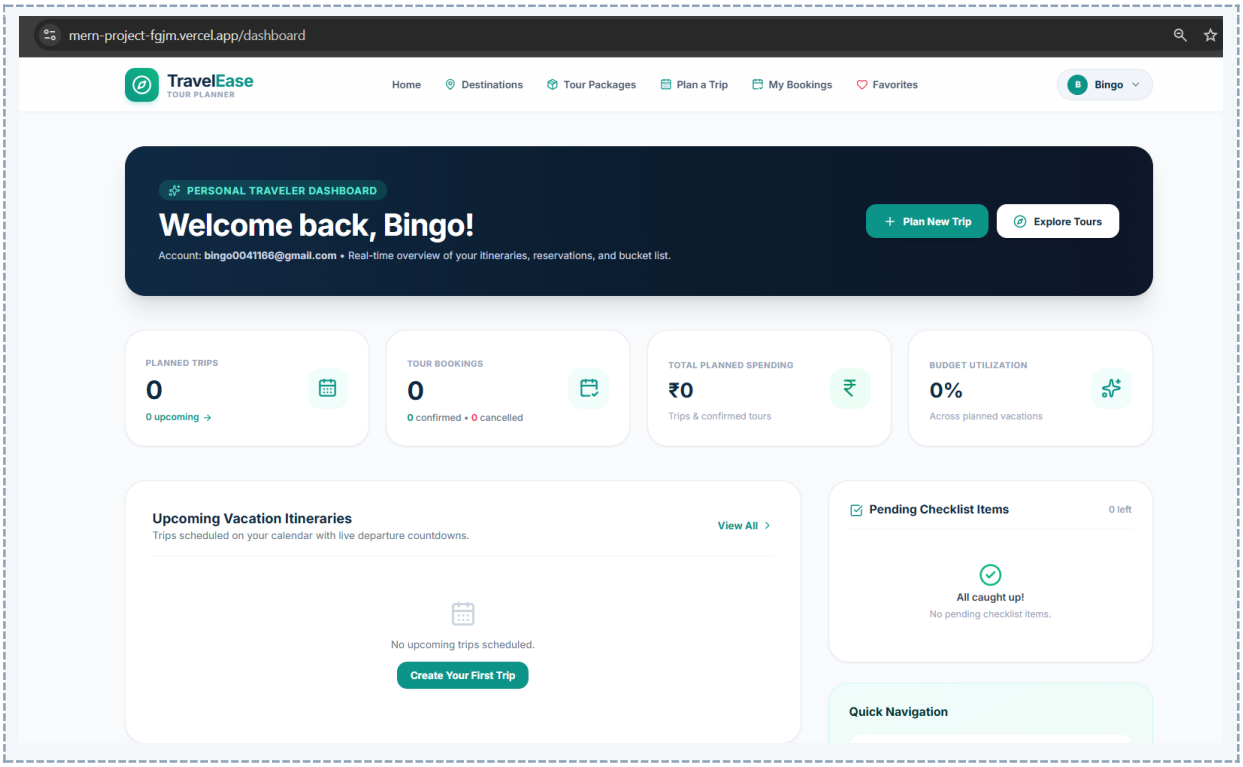


Figure 8: User Dashboard

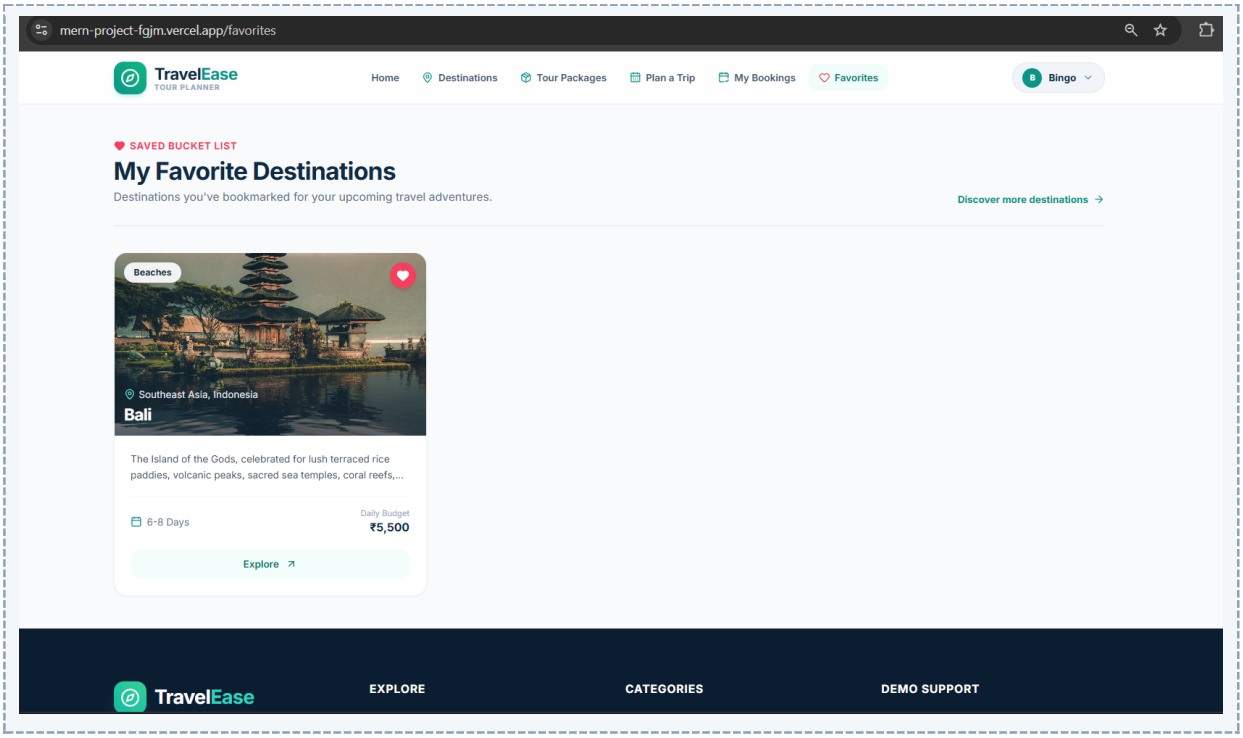


Figure 9: Favourite Destinations

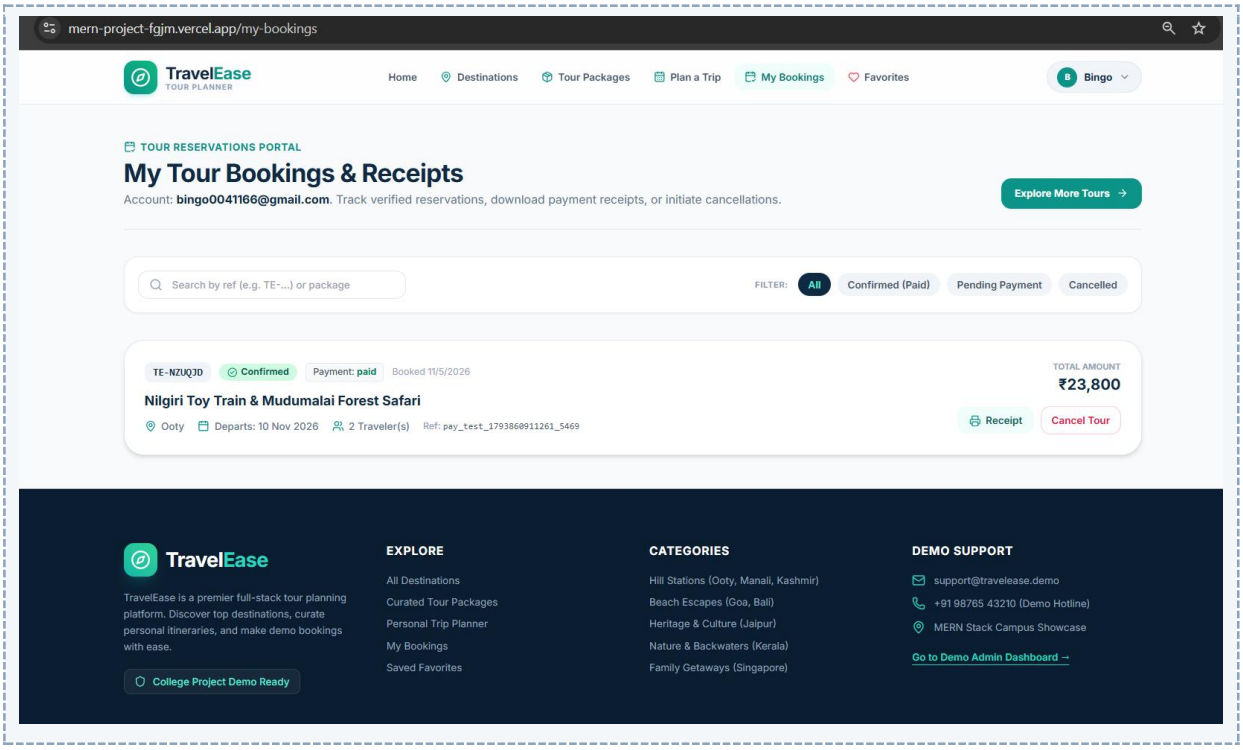


Figure 10: My Trips

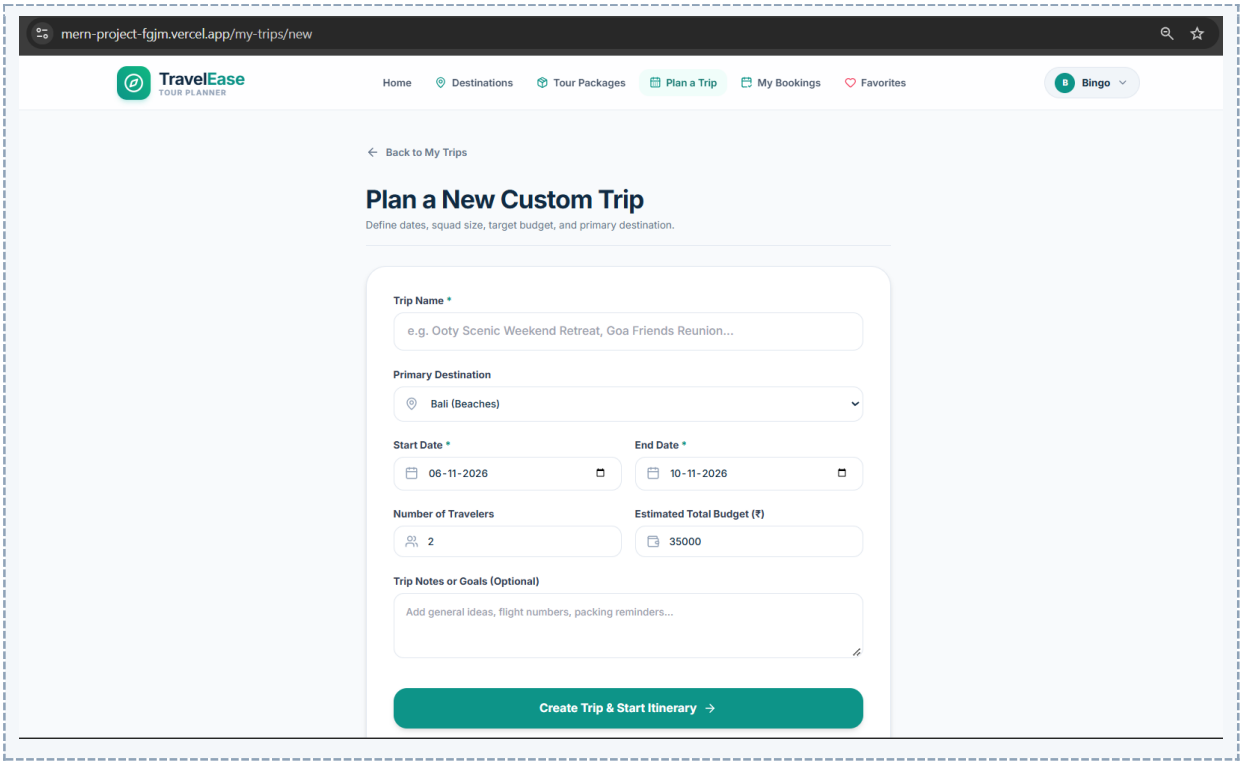
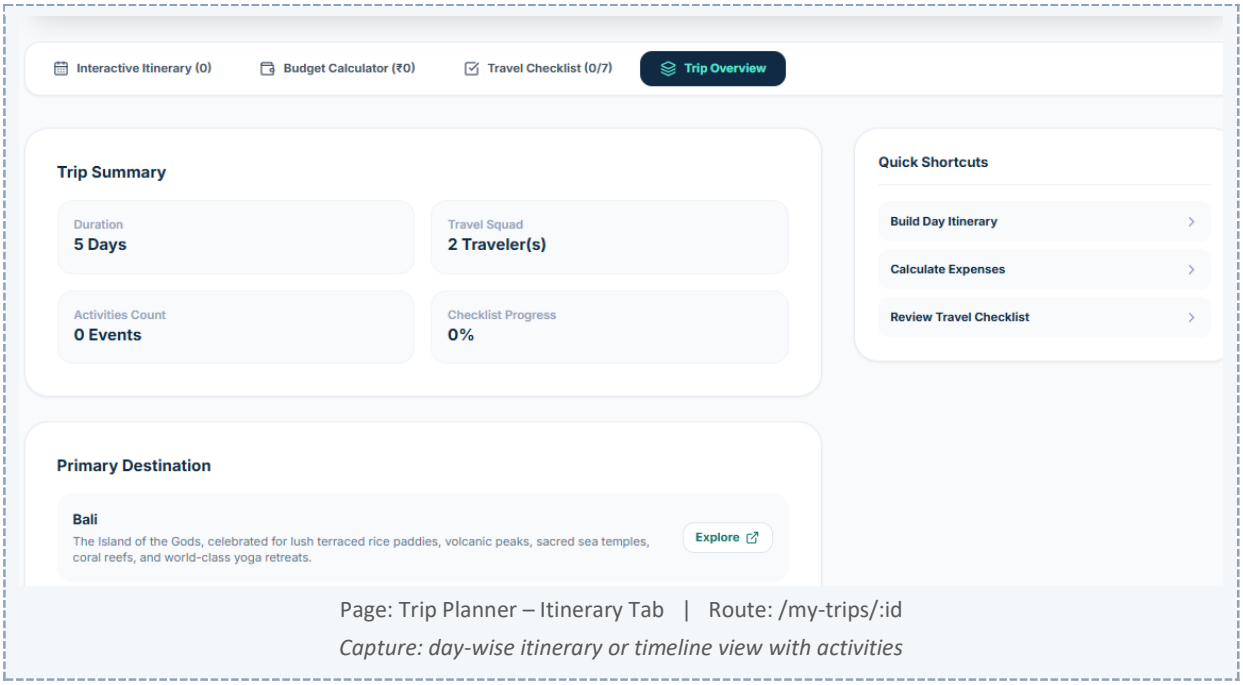


Figure 11: Create New Trip



Page: Trip Planner – Itinerary Tab | Route: /my-trips/:id

Capture: day-wise itinerary or timeline view with activities

Figure 12: Trip Planner – Itinerary Tab

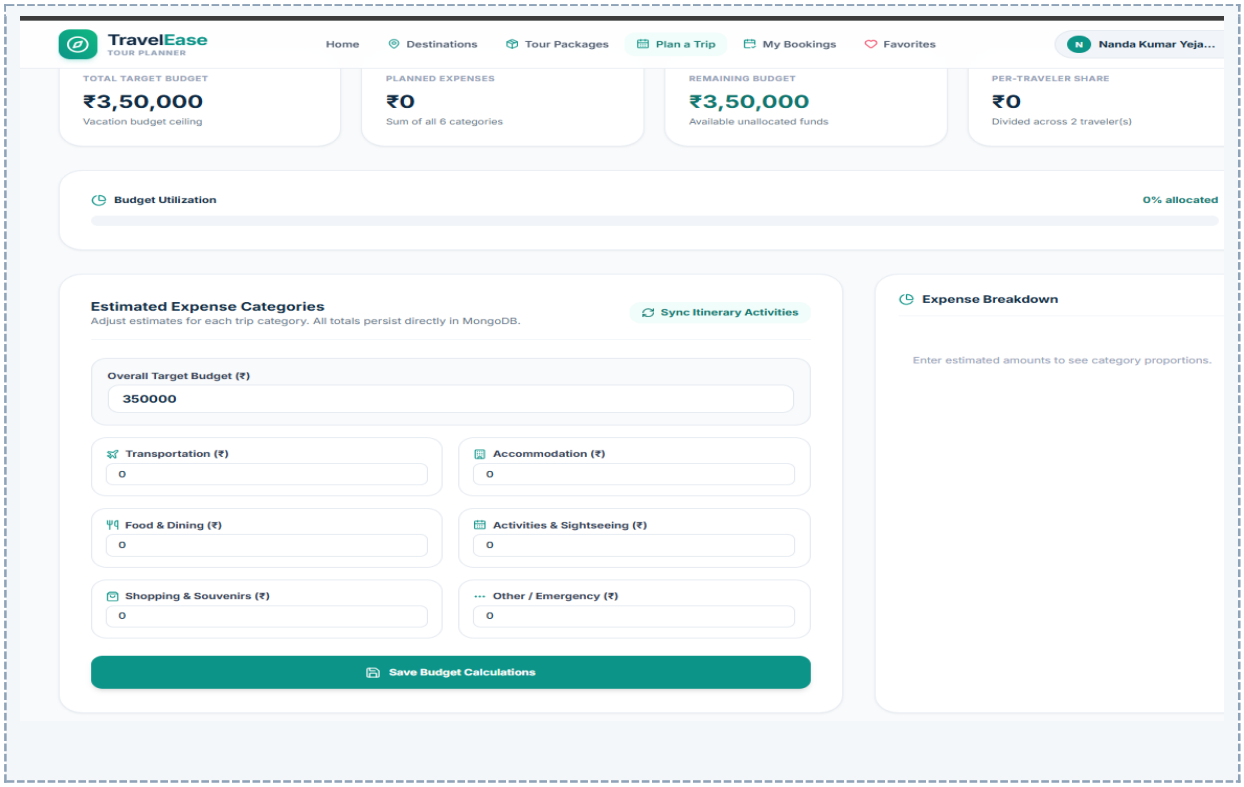


Figure 13: Trip Planner – Budget Tab

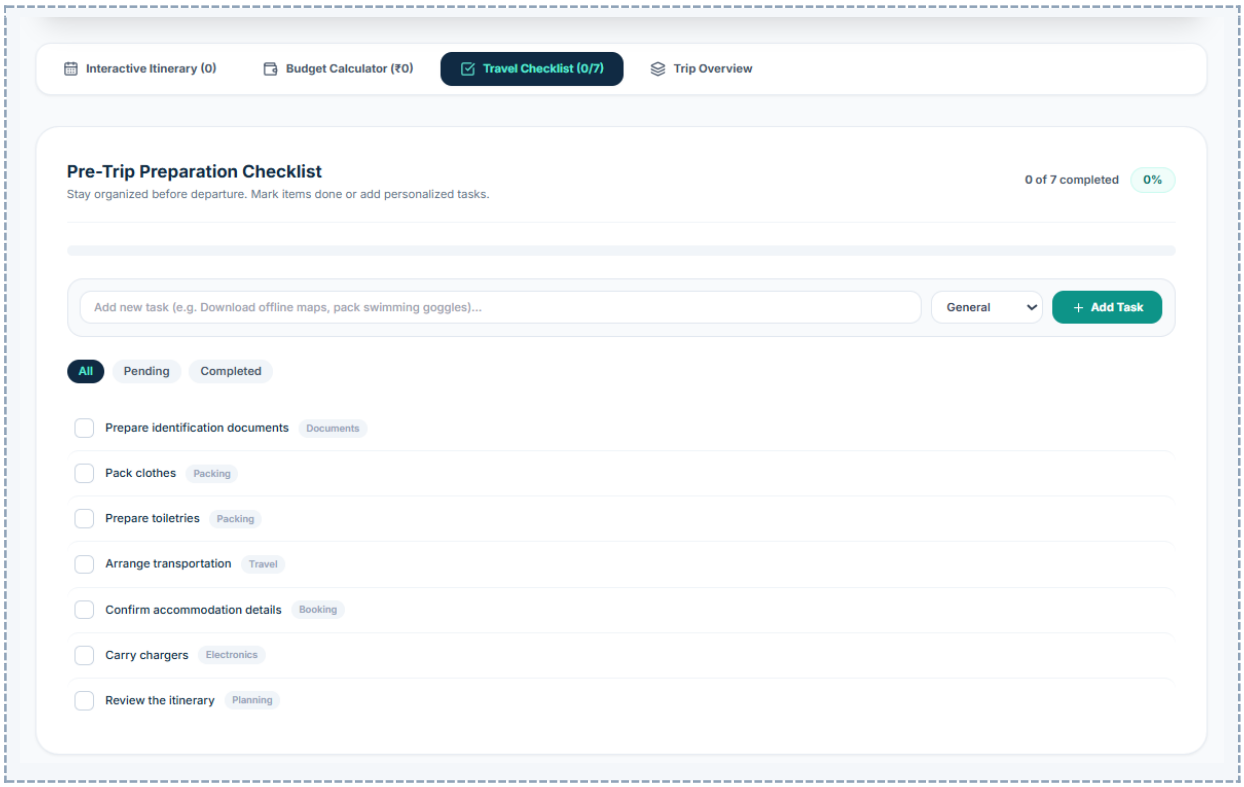


Figure 14: Trip Planner – Checklist Tab

5.2.4 Booking & Payment Pages

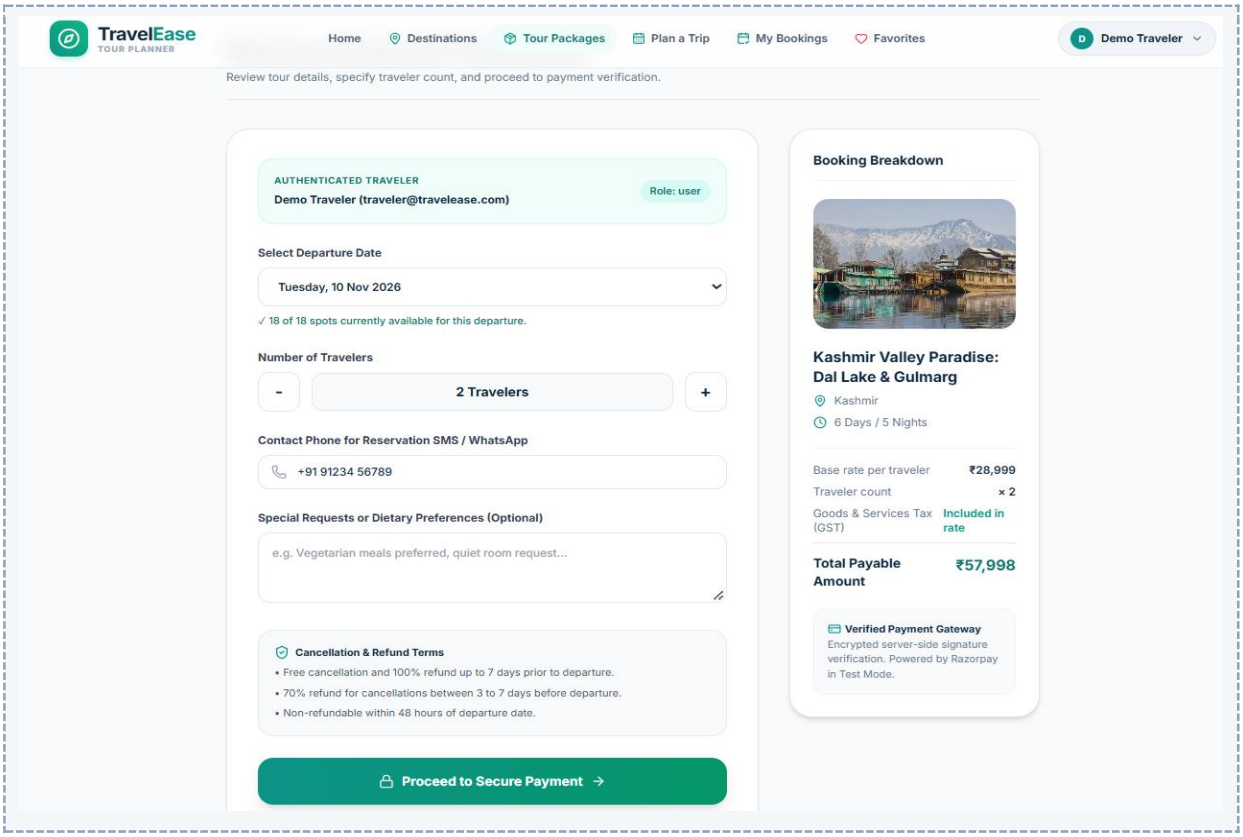


Figure 15: Review & Checkout

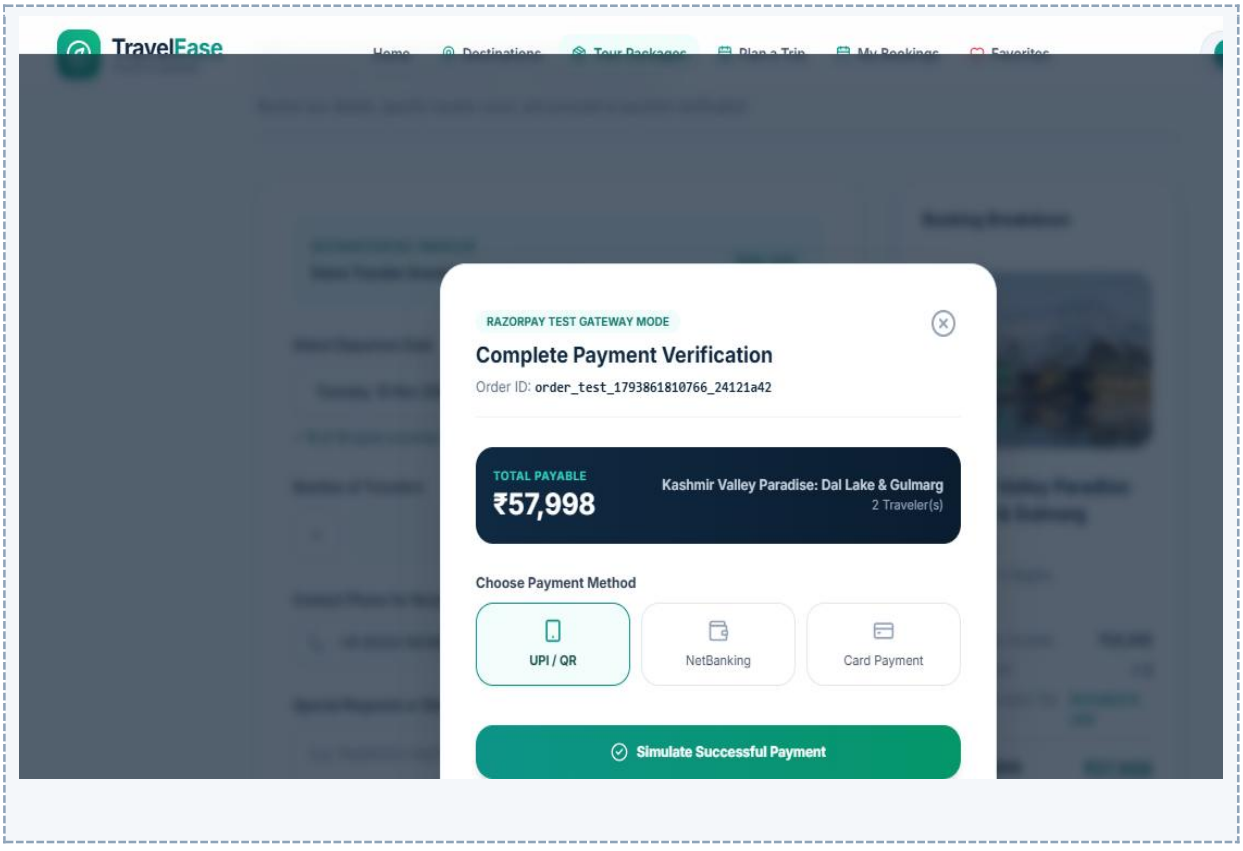


Figure 16: Payment Gateway Window

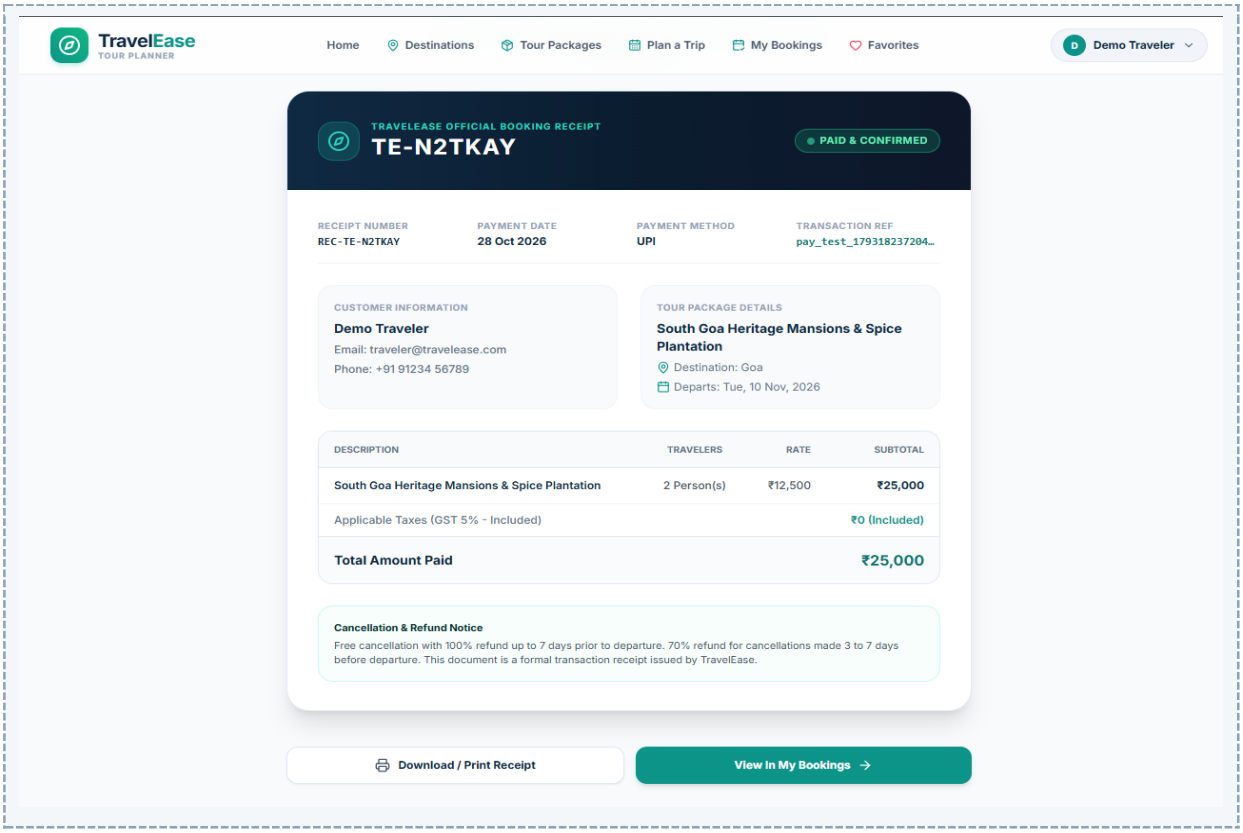


Figure 17: Booking Confirmation & Receipt

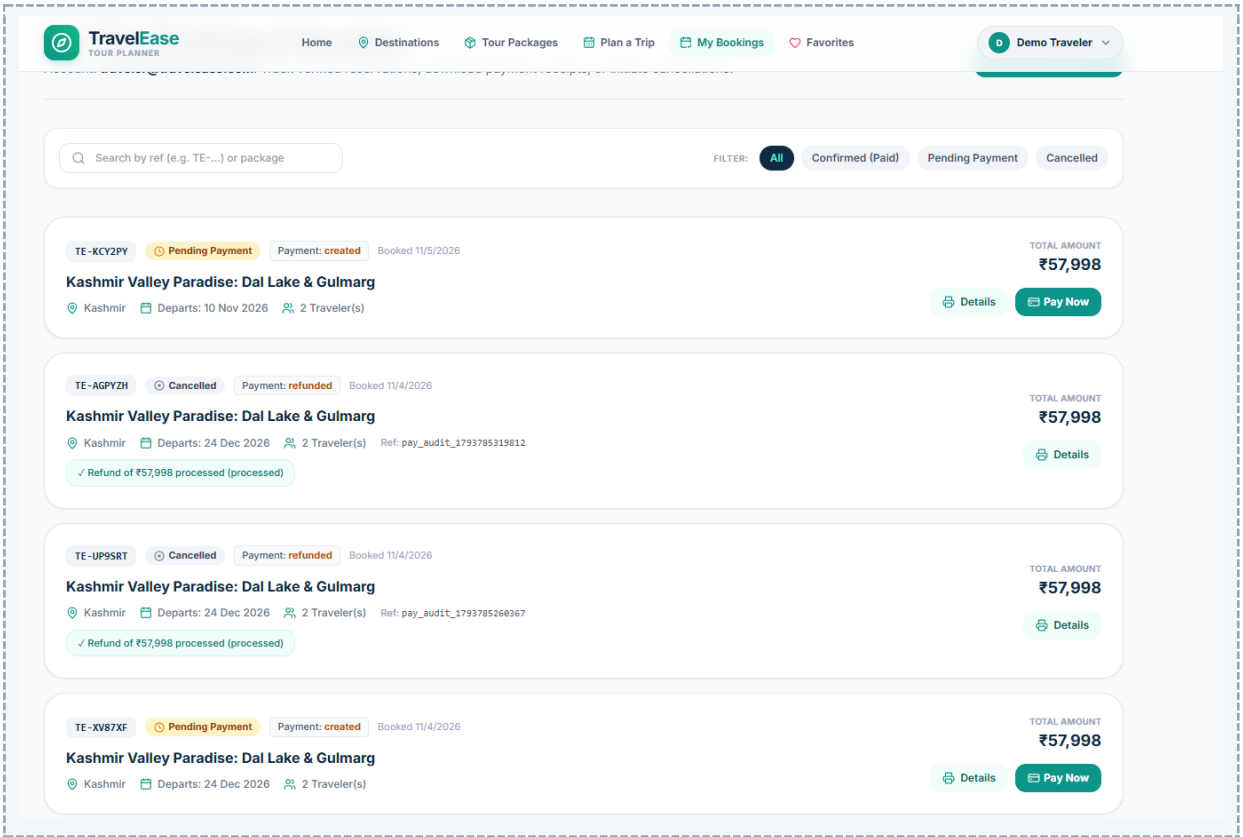


Figure 18: My Bookings

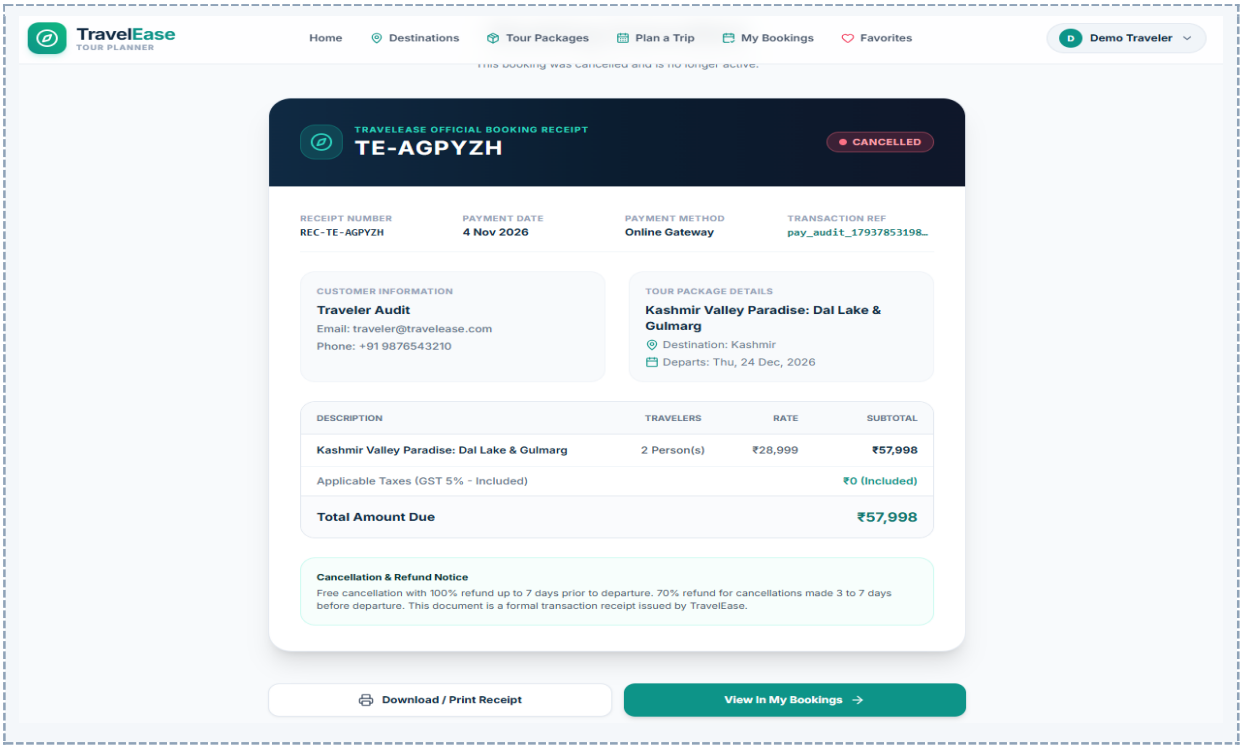


Figure 19: Booking Cancellation & Refund

5.2.5 Administrator Pages

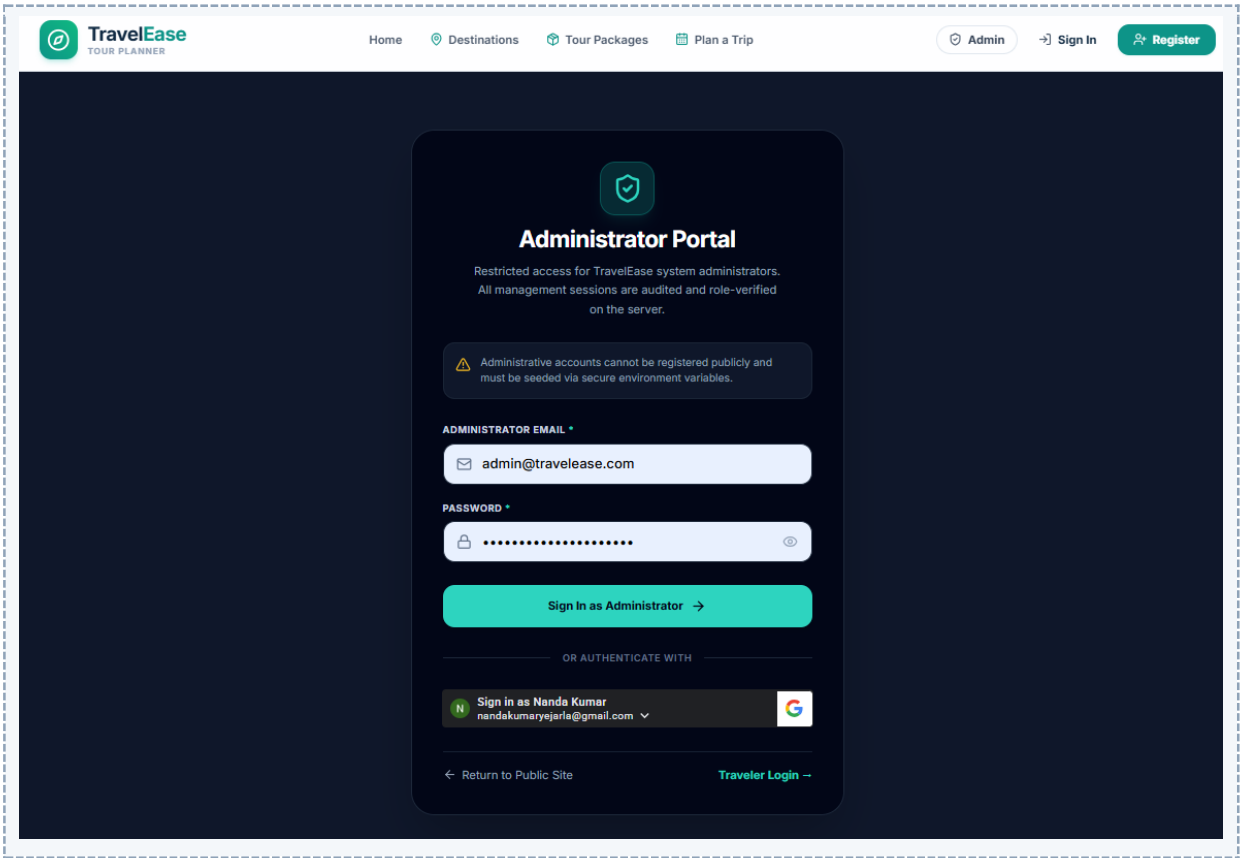


Figure 20: Administrator Login

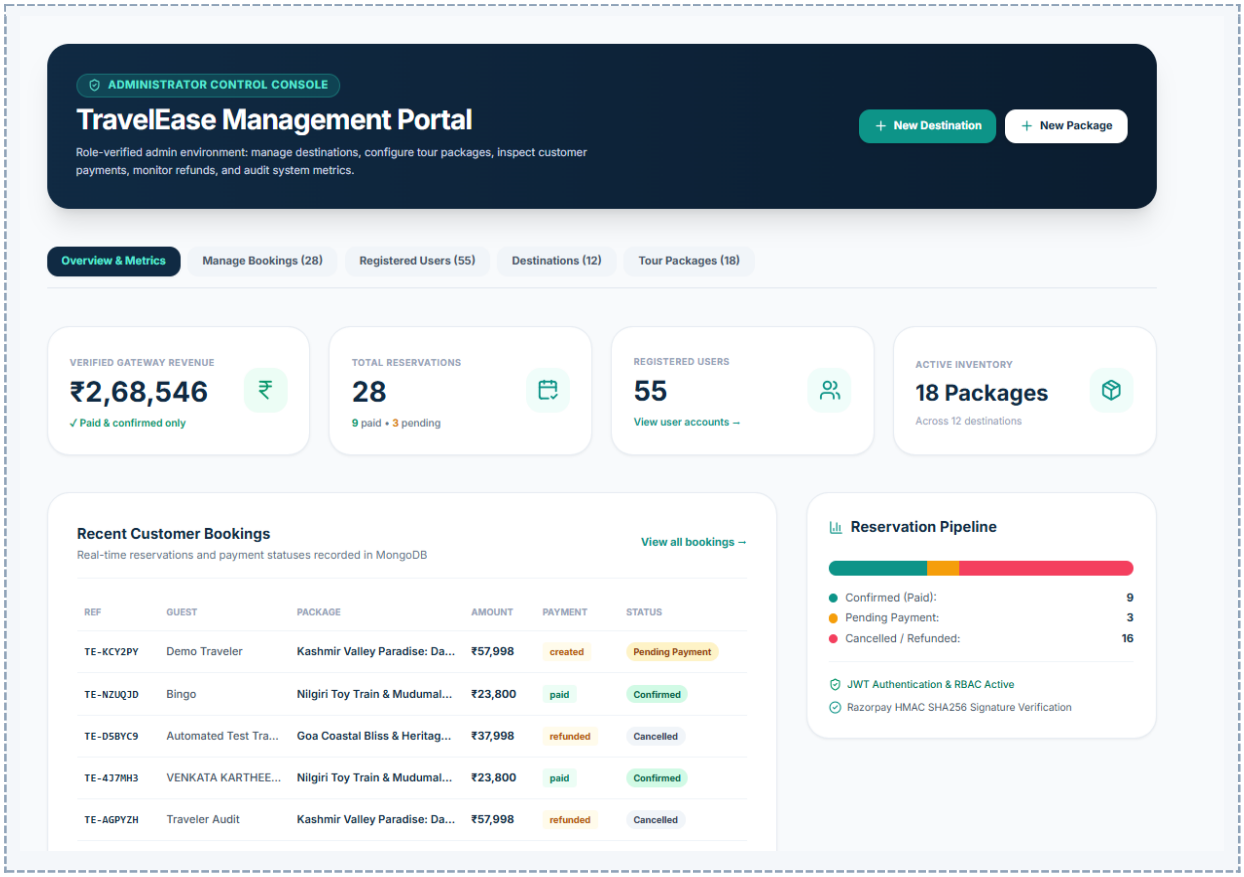


Figure 21: Administrator Dashboard

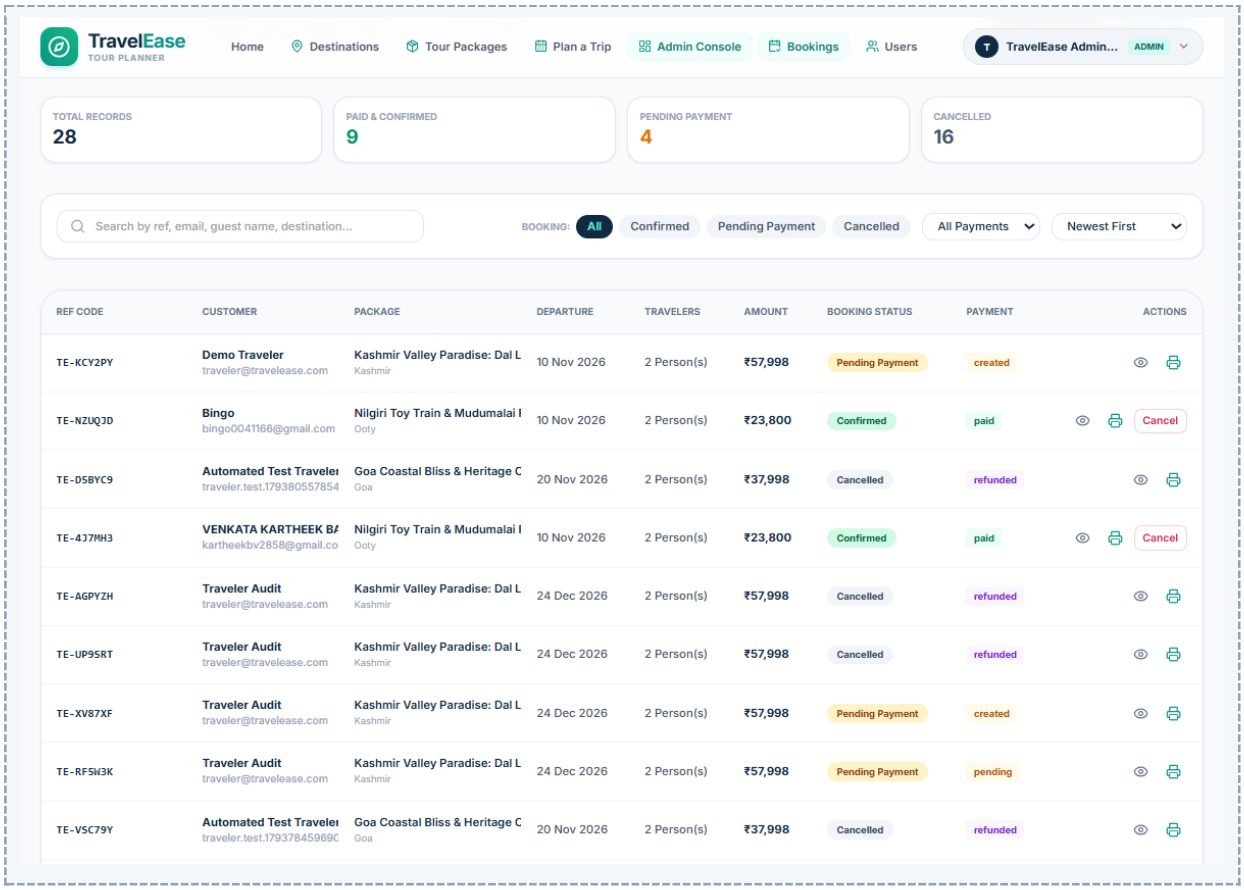


Figure 22: Bookings Management

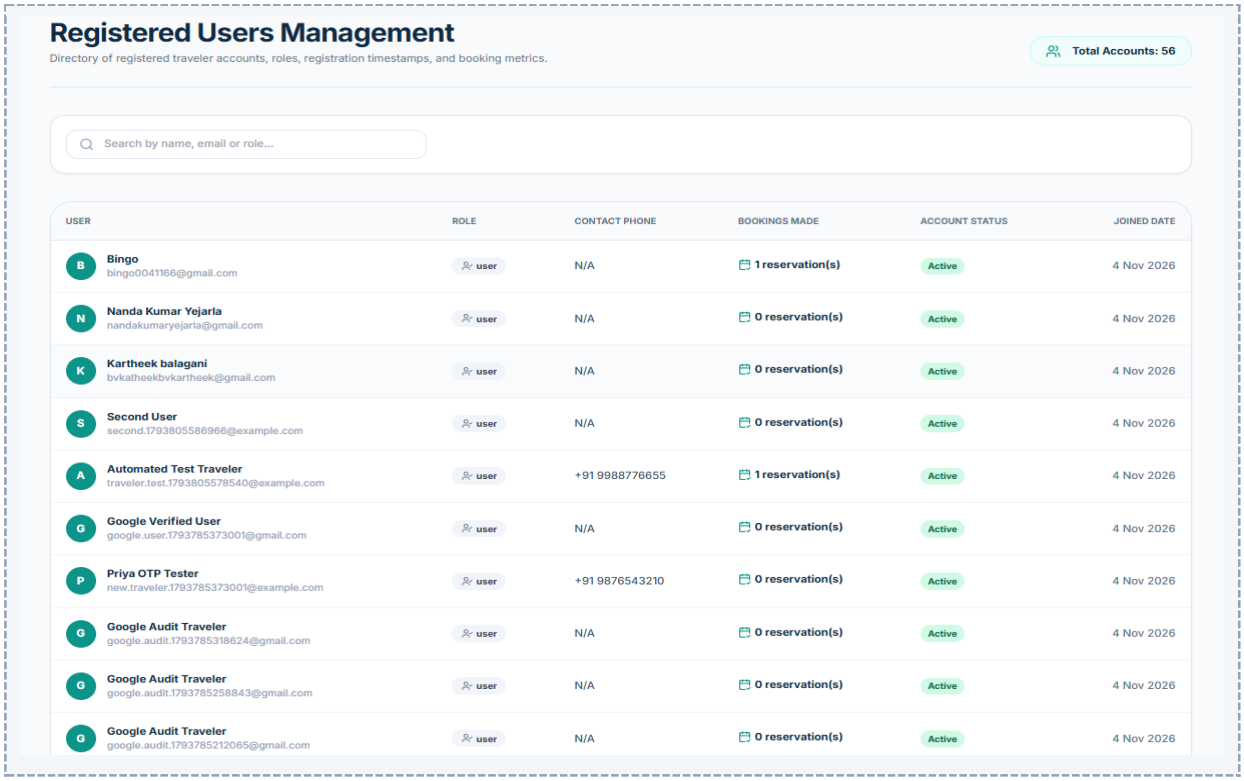


Figure 23: Registered Users

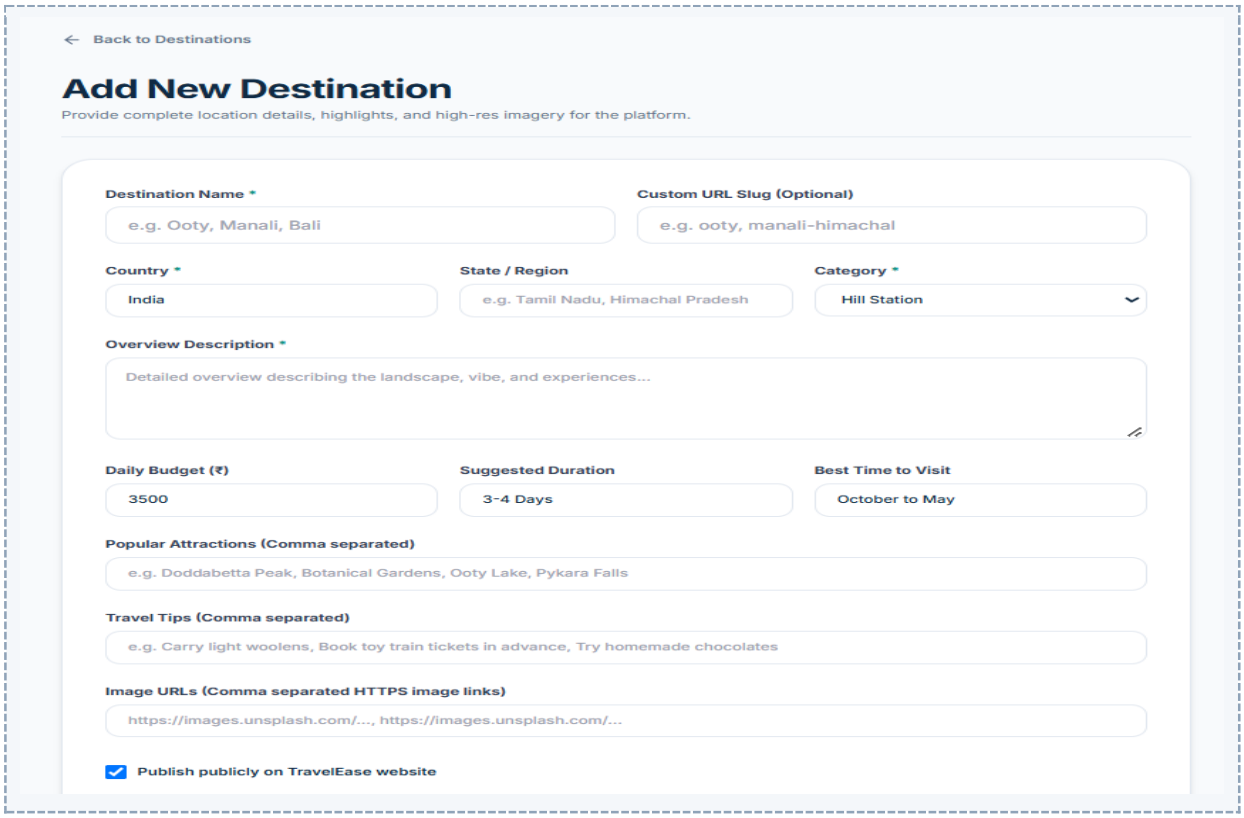


Figure 24: Destinations Management

5.3 Test Cases and Results

An automated end-to-end test script (server/test_e2e.js) runs against the API and database in test mode and checks authentication, role-based access, booking, payment verification, receipts and cancellation. It

is executed with “cd server && npm test”. The 21 checks defined in the script are listed below. Fill in the Result column after running the tests.

No.	Test Case	Expected Result	Result
1	User registration with valid details	Account is created and an OTP is requested (HTTP 201)	[Pass / Fail]
2	Role escalation during public registration	Registered user always has the “user” role	[Pass / Fail]
3	OTP verification	Account is activated and a JWT token is issued	[Pass / Fail]
4	Duplicate email registration	Request is rejected with an error	[Pass / Fail]
5	Password storage	Password is stored as a bcrypt hash, not plain text	[Pass / Fail]
6	Login with valid credentials	Login succeeds and a token is returned	[Pass / Fail]
7	Login with invalid credentials	Request is rejected (401)	[Pass / Fail]
8	Normal user calling administrator API	Access is denied (403 Forbidden)	[Pass / Fail]
9	Unauthenticated access to private endpoint	Access is denied (401 Unauthorized)	[Pass / Fail]
10	Administrator login	Login succeeds with role “admin”	[Pass / Fail]
11	Administrator platform statistics	Dashboard statistics are returned	[Pass / Fail]
12	Administrator user directory	List of registered users is returned	[Pass / Fail]
13	Booking creation	Booking is created with status “Pending Payment”	[Pass / Fail]
14	Server-side amount calculation	Total equals price per traveller × travellers	[Pass / Fail]
15	Payment order creation	An order ID is returned for the booking	[Pass / Fail]
16	Payment verification	Booking becomes “Confirmed” after verification	[Pass / Fail]
17	Duplicate payment verification (idempotency)	Booking stays “Confirmed” with no duplicate confirmation	[Pass / Fail]
18	Cross-user booking access	Another user’s booking is blocked (403 Forbidden)	[Pass / Fail]
19	Receipt retrieval	Receipt with the correct price breakdown is returned	[Pass / Fail]
20	Booking cancellation and refund evaluation	Booking becomes “Cancelled” and the refund is evaluated	[Pass / Fail]
21	Administrator booking inspection	Cancellation and payment details are visible to the administrator	[Pass / Fail]

Test Summary

Item	Details
Total test cases	21
Passed	[Enter count from your test run]
Failed	[Enter count from your test run]

Item	Details
Environment	Node.js, MongoDB (local or Atlas), NODE_ENV = test

The project also contains additional test scripts for OTP verification and authentication (test_otp_and_auth.js), Google sign-in (test_google_auth.js) and database operations on MongoDB Atlas (test_atlas_dynamic_operations.js).

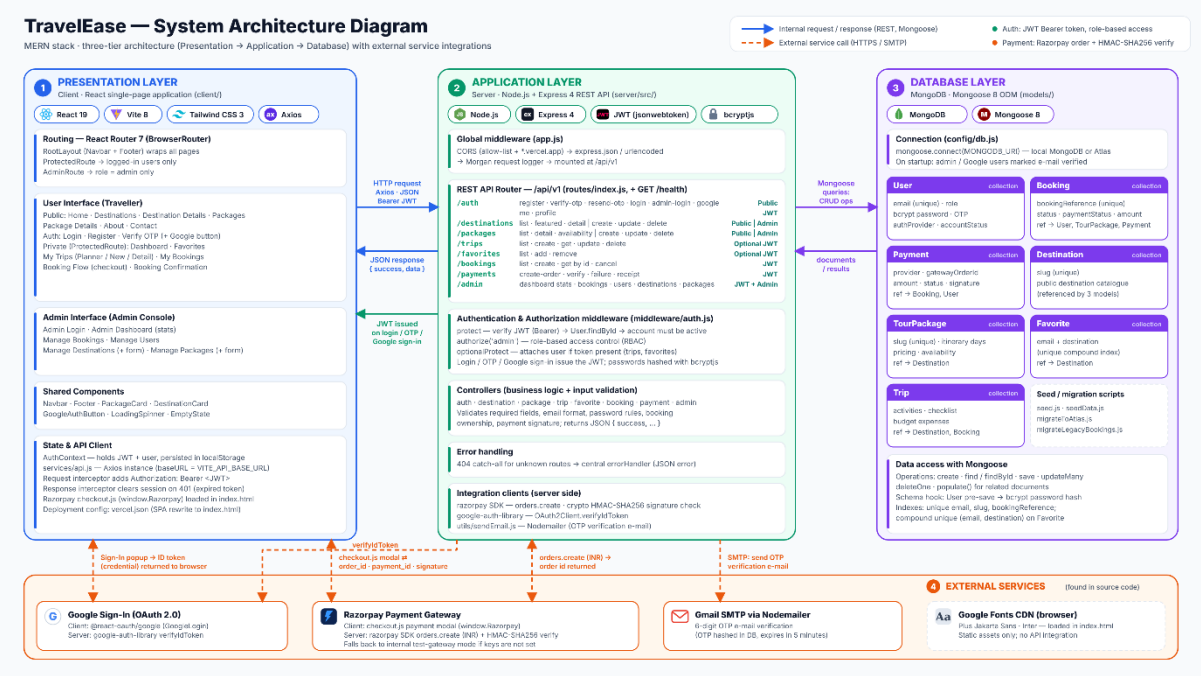


Figure 25: Architecture Diagram

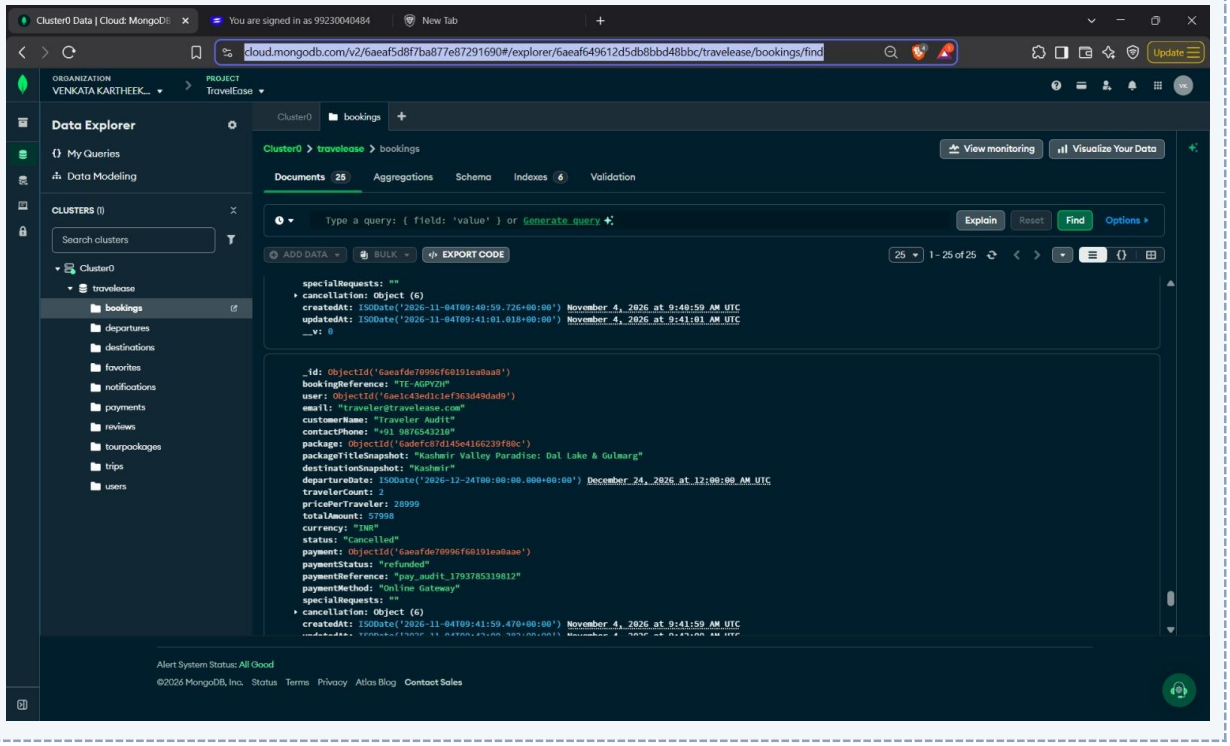


Figure 26: MongoDB Collections

6. Conclusion & Future Scope

Conclusion

TravelEase brings destination discovery, tour booking, online payment verification and personal trip planning into a single MERN-stack application. The project demonstrates secure authentication with OTP verification and JWT, role-based access control, server-side price calculation, signature-based payment verification, a refund policy driven by departure date, and an administrator console with verified revenue tracking. The automated test script checks the main backend flows, and the responsive interface lets users complete the whole journey from browsing to a confirmed booking.

Future Scope

Future improvements may include enforcing ownership checks on trips and favourites using the logged-in user's identity instead of an email parameter, adding rate limiting and input sanitisation on public endpoints, and moving session tokens to secure HTTP-only cookies with refresh tokens. On the product side, the platform can be extended with real user reviews and ratings, booking confirmation emails, live Razorpay production payments with real refunds, multi-currency pricing for international packages, map and weather integration in the trip planner, and analytics charts for administrators. Automated front-end tests and a continuous integration pipeline would also make the project easier to maintain.